

R.M.K GROUP OF ENGINEERING INSTITUTIONS



R.M.K
GROUP OF
INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS



R.M.K
GROUP OF
INSTITUTIONS



Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

24CS402

Design and Analysis of Algorithms

Batch/Year: 2024-2028/II

Created by:

Dr. S. Thanga Ramya, Professor & Head, CSD, RMKEC
Dr. R.Chithambaramani, Assoc. Professor, CSE, RMKEC
Ms. P. Baby Shamini, Asst. Professor, CSE, RMKEC
Ms. S. Kanimozhi, Asst. Professor, CSE, RMKEC
Dr. S.Selvakanmani, Assoc. Professor, IT, RMKEC
Dr.T.Mahalingam, Assoc. Professor, IT, RMKEC
Dr.A.Gnanasekar, Assoc. Professor, CSE, RMD
Mr.K.Mohanasundaram, Asst. Professor, CSE, RMD
Mr.D.Jayakumar, Asst. Professor, CSE, RMD
Dr.J.Sherine Glory, Assoc. Professor, CSE, RMD
Mrs.D.M.Kalaiselvi, Asst. Professor, CSE, RMD
Ms. M. Sindhu, Asst. Professor, CSBS, RMKEC
Dr. V. R. Kavitha, Assoc. Professor, CSBS, RMKEC
Ms. K. Amuthachenthiru, Asst. Professor, AIML, RMDEC
Mrs.N.Arockia Rosy, Assoc. Professor, IT, RMDEC
Mrs.K.Anandhi, Asst. Professor, IT, RMDEC
Dr. Prasath R, Assoc. Professor, CSE, RMKCET
Dr Indra G, Assoc. Professor, CSE, RMKCET
Dr.Vinoth R, Assoc. Professor, CSE, RMKCET
Ms.S.Anitha Rajathi, Asst. Professor, CSBS, RMDEC
Ms.R.Monica Lakshmi, Asst. Professor, CSBS, RMDEC
Dr.Gladiss Merlin, Assoc. Professor, ADS, RMKEC
Ms.Vimala Josephine, Asst. Professor, ADS, RMKEC
Dr.Cornelius K, Assoc. Professor, ADS, RMKCET
Ms. Uma Maheshwari, Asst. Professor, CSD, RMKEC
Dr. Vendhan Duraisamy, Asst. Professor, CSE(CS), RMKEC



Table of Contents

Sl. No.	Topics	Page No.
1.	Contents	5
2.	Course Objectives	6
3.	Pre Requisites (Course Name with Code)	8
4.	Syllabus (With Subject Code, Name, LTPC details)	10
5.	Course Outcomes (6)	12
6.	CO-PO/PSO Mapping	14
7.	Lecture Plan (S.No., Topic, No. of Periods, Proposed date, Actual Lecture Date, pertaining CO, Taxonomy level, Mode of Delivery)	16
8.	Activity based learning	18
9.	Lecture Notes (with Links to Videos, e-book reference, PPTs, Quiz and any other learning materials)	20-61
10.	Assignments (For higher level learning and Evaluation - Examples: Case study, Comprehensive design, etc.,)	62
11.	Part A Q & A (with K level and CO)	66-74
12.	Part B Qs (with K level and CO)	75
13.	GATE Questions	77
14.	Supportive online Certification courses (NPTEL, Swayam, Coursera, Udemy, etc.,)	80
15..	Real time Applications in day to day life and to Industry	82
16.	Contents beyond the Syllabus (COE related Value added courses)	84
17.	Assessment Schedule (Proposed Date & Actual Date)	88
18.	Prescribed Text Books & Reference Books	89
19.	Mini Project	91



R.M.K.
GROUP OF
INSTITUTIONS

Course Objectives

Course Objectives

- **Critically analyse the efficiency of alternative algorithmic solutions for the same problem**
- **Illustrate brute force and divide and conquer design techniques.**
- **Explain dynamic programming for solving various problems.**
- **Apply greedy technique and iterative improvement technique to solve optimization problems**
- **Examine the limitations of algorithmic power and handling it in different problems.**



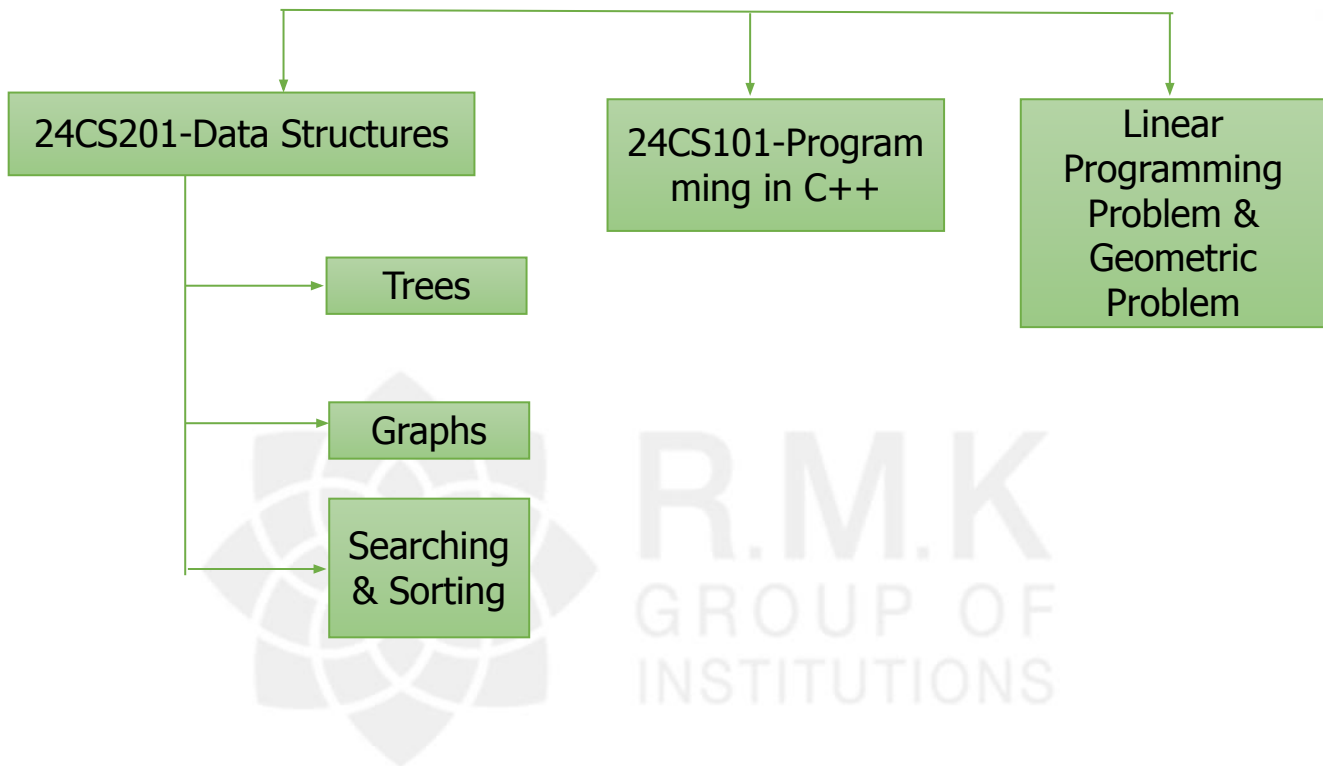
R.M.K.
GROUP OF
INSTITUTIONS



R.M.K.
GROUP OF
INSTITUTIONS

PRE REQUISITES

Prerequisites





R.M.K.
GROUP OF
INSTITUTIONS

Syllabus

Syllabus

24CS402-DESIGN AND ANALYSIS OF ALGORITHMS

L T P C

3 0 2 4

9+6

Unit I : INTRODUCTION

Notion of an Algorithm – Fundamentals of Algorithmic Problem Solving – Fundamentals of the Analysis of Algorithmic Efficiency – Asymptotic Notations and their properties. Analysis Framework – Mathematical analysis for Recursive and Non-recursive algorithms.

List of Exercise/Experiments:

1. Perform the recursive algorithm analysis.
2. Perform the non-recursive algorithm analysis.

Unit II : BRUTE FORCE AND DIVIDE AND CONQUER 9+6

Brute Force – String Matching – Exhaustive Search - Knapsack Problem. Divide and Conquer Methodology – Binary Search – Merge sort- Quick sort – Multiplication of Large Integers – Closest-Pair and Convex Hull Problems. -Transform and Conquer Method: Heap Sort

List of Exercise/Experiments:

1. Write a program to search an element using binary search
2. Write a program to sort the elements using merge sort and find time complexity.

Unit III : DYNAMIC PROGRAMMING 9+6

Dynamic programming – Principle of optimality - Floyd's algorithm – Multi stage graph - Optimal Binary Search Trees – Longest common subsequence - Matrix-chain multiplication – Travelling Salesperson Problem – Knapsack Problem and Memory functions.

List of Exercise/Experiments:

1. Solve Floyd's algorithm
2. Write a program to find the longest common subsequence

Unit IV : GREEDY TECHNIQUE AND ITERATIVE IMPROVEMENT 9+6

Greedy Technique - Prim's algorithm and Kruskal's Algorithm, Huffman Trees. The Maximum-Flow Problem – Maximum Matching in Bipartite Graphs- The Stable marriage Problem.

List of Exercise/Experiments:

1. Write a program to find minimum spanning tree using Prim's algorithm
2. Implement Kruskal's algorithm to find minimum spanning tree

Unit V : BACKTRACKING AND BRANCH AND BOUND 9+6

P, NP, NP - Complete and NP Hard Problems. Backtracking – N-Queen problem - Subset Sum Problem. Branch and Bound – LIFO Search and FIFO search - Assignment problem – Knapsack Problem - Approximation Algorithms for NP-Hard Problems – Travelling Salesman problem

List of Exercise/Experiments:

1. Write a program to implement sum of subset problem.
2. Solve knapsack problem using branch and bound technique



R.M.K.
GROUP OF
INSTITUTIONS

Course Outcomes

Course Outcomes

CO#	COs	K Level
CO1	Understand the different algorithm design paradigms	K2
CO2	Design algorithms for real world problems using algorithmic design techniques	K4
CO3	Analyse the efficiency of simple recursive and non-recursive algorithms	K4
CO4	Analyse the algorithm's worst, best and average case behaviour in terms of time and space.	K4
CO5	Understand the approximation algorithms for solving NP Hard problems	K2
CO6	Solve the problems by selecting suitable algorithmic design techniques.	K3

Knowledge Level	Description
K6	Evaluation
K5	Synthesis
K4	Analysis
K3	Application
K2	Comprehension
K1	Knowledge



R.M.K.
GROUP OF
INSTITUTIONS

CO – PO/PSO Mapping

CO – PO /PSO Mapping Matrix

CO #	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
C01	3	3	-		1	-	2	2	2	-	2	3	2	-
C02	3	2	-	2	2	-	2	2	2	-	2	3	2	-
C03	3	3	2	-	2	-	2	2	2	-	2	3	2	-
C04	3	2	2	2	-	-	2	2	2	-	2	3	3	-
C05	3	2	2	-	2	-	2	2	2	-	2	3	2	-
C06	3	2	2	-	-	-	2	2	2	-	2	2	2	-



R.M.K.
GROUP OF
INSTITUTIONS



R.M.K.
GROUP OF
INSTITUTIONS

Lecture Plan

Unit I

Lecture Plan – Unit 1 -Introduction

Sl. No	Topic	Number of Periods	Proposed Date	Actual Lecture Date	CO	Taxonomy Level	Mode of Delivery
1	Notion of an Algorithm –	1			CO1	K2	Chalk & Talk
2	Fundamentals of Algorithmic Problem Solving	1			CO1	K2	Chalk & Talk
3	Fundamentals of the Analysis of Algorithmic Efficiency	1			CO1	K2	Chalk & Talk
4	Asymptotic Notations and their properties	1			CO1	K2	Chalk & Talk
5	Analysis Framework	1			CO1	K2	Chalk & Talk
6	Mathematical analysis for Non-Recursive Algorithms	1			CO1	K3	Chalk & Talk
7	Mathematical analysis for Non-Recursive Algorithms	1			CO1	K3	Chalk & Talk, Demo
8	Mathematical analysis for Recursive Algorithms	1			CO1	K3	Chalk & Talk
9	Mathematical analysis for Recursive Algorithms	1			CO1	K3	Chalk & Talk, Demo



R.M.K.
GROUP OF
INSTITUTIONS

Activity Based Learning

Unit I

Activity Based Learning

Flash cards

<https://www.brainscape.com/p/5ANHD-LH-D6R7A>

Time Complexity Calculation

<https://www.timecomplexity.ai/>

Quiz-Time Complexity

<https://www.interviewbit.com/courses/programming/time-complexity/>



R.M.K.
GROUP OF
INSTITUTIONS

Lecture Notes – Unit 1

UNIT I INTRODUCTION

Sl. No.	Contents	Page No.
1	Need to study algorithms	22
2	Notion of Algorithms	23
3	Fundamentals of Algorithm Problem Solving	28
4	Fundamentals of the analysis of algorithm efficiency	35
5	Asymptotic Notations and their properties.	44
6	Mathematical analysis of Non-Recursive Algorithms	50
7	Mathematical analysis of Recursive Algorithms	56



R.M.K.
GROUP OF
INSTITUTIONS

Unit I INTRODUCTION

Need to study algorithms:

- There are both practical and theoretical reasons to study algorithms.
- From the practical point of view
 - to know a standard set of important algorithms from different areas of computing.
 - able to design new algorithms and analyze their efficiency.
- From theoretical point of view
 - study of algorithms, sometimes called algorithmics, has come to be recognized as the cornerstone of computer science.
- Another reason for studying algorithms is their usefulness in developing analytical skills.
- Algorithms can be seen as special kinds of solutions to problems-not answers but rather precisely defined procedures for getting answers.
- Well trained computer science professionals knows how to deal with algorithms:
 - how to construct them.
 - manipulate them.
 - understand them.
 - analyze them.

The knowledge is preparation for much more than writing good computer programs.

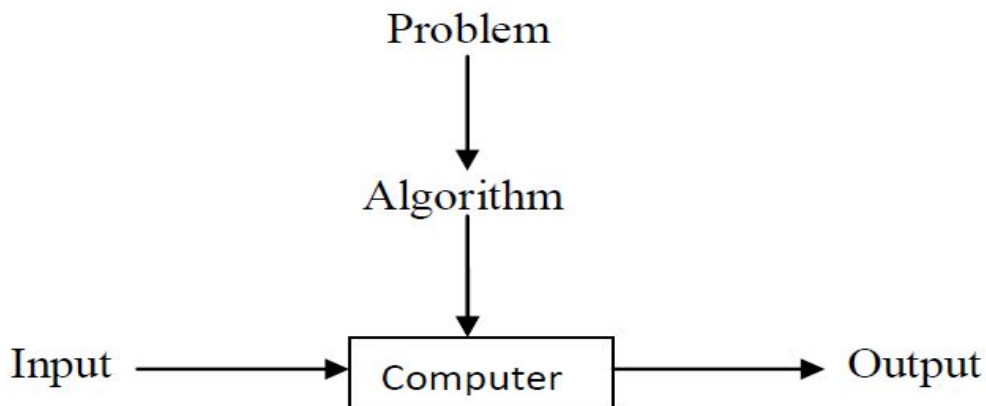
- Actually, a person does not really understand something until after teaching it to someone else. Similarly, a person does not understand until after teaching it to a computer.
 - i.e., Expressing it as an algorithms.

Unit I INTRODUCTION

1.1. Notion of algorithms:

What is an algorithm?

An algorithm is a sequence of unambiguous instructions for solving a problem. i.e., For obtaining a required output for any legitimate input in a finite amount of time. This definition can be illustrated by a simple diagram in Fig 1.



•As examples illustrating the notation of algorithm, we consider three methods for solving the same problem.

Ex: Computing the greatest common divisor of two integers.

•These examples will help us to illustrate several important points.

-The non-ambiguity requirement for each step of an algorithm cannot be compromised.

-The range of inputs for which an algorithm works has to be specified carefully.

-The same algorithm can be represented in several different ways.

-Several algorithms for solving the same problem may exist.

- Algorithms for the same problem can be based on very different ideas and can solve the problem with different speeds.

Unit I INTRODUCTION

Problem:

Greatest common divisor of two non-negative, non-both zero integers m and n , denoted $\text{gcd}(m,n)$.

$\text{gcd}(m,n)$ is defined as the largest integer that divides both m and n evenly, i.e., with a remainder zero.

I Method:

Euclid's algorithm is based on $\text{gcd}(m,n) = \text{gcd}(n, m \bmod n)$ until $m \bmod n$ is equal to $\emptyset$, the value of m is also the gcd of the initial m and n .

Ex: $\text{gcd}(60,24)$ can be computed as $\text{gcd}(60,24) = \text{gcd}(24,12) = \text{gcd}(12,0) = 12$

Structured description of the algorithm:

Step 1: If $n = 0$ return the value of n as the answer and stop; otherwise proceed to step 2.

Step 2: Divide m by n and assign the value of the remainder to r .

Step 3: Assign the value of n to m and the value of r to n . Goto step 1.

Algorithm: Euclid(m,n)

//Input: Two non-negative, not both-zero integers m and n .

//Output: gcd of m and n .

While $n \neq 0$ do

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

Unit I INTRODUCTION

How this loop terminates:

From the observation, the second number of the pair gets smaller with each iteration and it cannot be negative.

Hence the value of the second number in the pair eventually becomes 0, and the algorithm stops.

II Method:

Common divisor cannot be greater than the smaller of these numbers.

(i.e) $t = \min\{m,n\}$, we check whether t divides both m & n .

if it does, t is the answer.

If it does not, decreases t by 1 and try again.

Ex: $m = 60$ $n = 24$

$$\min(60,24) = 24$$

The algorithm will try first 24, then 23 and so in until it reaches 12, where it stops.

Structured algorithm:

Step 1: Assign the value of $\min\{m,n\}$ to t .

Step 2: Divide m by t . If the remainder of this division 0, goto step 3; Otherwise goto step 4.

Step 3: Divide n by t . If the remainder of this division is 0, return the value of t as the answer and stop; Otherwise proceed to step 4.

Step 4: Decrease the value of t by 1. Goto step 2.

This algorithm does not work correctly, when one of its input numbers is zero.

This example illustrates, the importance of specifying the range of an algorithm's inputs explicitly and carefully.

Unit I INTRODUCTION

III Method:

Structured algorithm: Middle-school procedure

Step 1: Find the prime factors of m.

Step 2: Find the prime factors of n.

Step 3: Identify all the common factors in the two prime expansions found in step 1 and step 2.

Step 4: Compute the product of the all the common factors and return it as the greatest common divisor of the numbers given.

Ex: 60 and 24

$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$

$$\gcd(60, 24) = 2 \cdot 2 \cdot 3 = 12$$

- This algorithm is much more complex and slower than Euclid's algorithm.
- It does not qualify, in the form presented, as a legitimate algorithm. Because the prime factorization steps are not defined unambiguously. They require list of prime numbers.
- Simple algorithm for generating consecutive primes not exceeding any given integer n.
- The algorithm
 - Starts by initializing a list of prime candidates with consecutive integers from 2 to n.
 - On the I iteration of the algorithm, it eliminates from the list all multiples of 2. i.e., 4, 6 and so on.
 - Then it moves to the next item on the list, which is 3 and eliminates its multiples.

Unit I INTRODUCTION

- The algorithm continues in this fashion until no more numbers can be eliminated from the list.
- The remaining integers of the list are the primes needed.

Ex: Finding the list of primes not exceeding $n = 25$.

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

2 3 5 7 9 11 13 15 17 19 21 23 25

2 3 5 7 11 13 17 19 23 25

2 3 5 7 11 13 17 19 23

Number of passes is needed because

- they would eliminate numbers already eliminated on previous iterations of the algorithm.
- The remaining numbers on the list are the consecutive primes less than or equal to 25.
- If p is a number whose multiples are being eliminated on the current pass, then the first multiple we should consider is

$$p \cdot p \leq n \quad \Rightarrow \quad p \text{ cannot exceed } \sqrt{n}$$

$$p^2 = n$$

$$p = \sqrt{n}$$

Unit I INTRODUCTION

- **Algorithm: Sieve(n)**

```
for p ← 2 to n do A[p] ← p
  for p ← 2 to  $\lfloor \sqrt{n} \rfloor$  do
    if A[p] ≠ 0
      j ← p*p
      while j < n do
        A[j] ← 0
      j ← j+p
    i ← 0
  for p ← 2 to n do
    if A[p] ≠ 0
      L[i] ← A[p]
      i ← i+1
return L
```

So, we incorporate the sieve into middle school procedure to get a legitimate algorithm for computing gcd.

1.2. Fundamentals of algorithm problem solving:

Sequence of steps in designing & analyzing an algorithm.

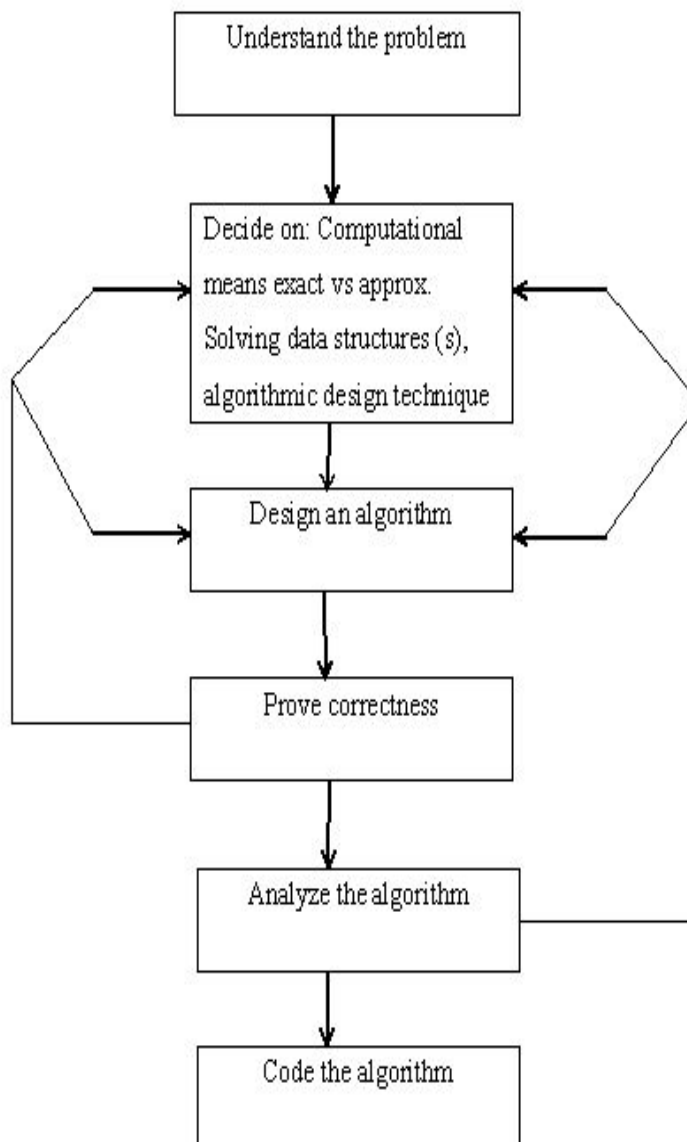
Understanding the problem:

- Before designing an algorithm is to understand completely the problem.
- Read the problems description and ask questions or doubt about the problems.
- Solve the few small examples by hand.
- Think about special cases and ask questions again if needed.
- There are few types of problems that arise in computing applications quite often.
- There are known algorithms for solving it. It might help us to understand how such an algorithm works and know its strengths and weakness.

Unit I INTRODUCTION

- There will not be readily available algorithm for some problems and we have to design on our own.
- An input to an algorithm specifies an instance of the problem the algorithm solves.
- It is very important to specify exactly the range of instances the algorithm needs to handle.
- If we fail to do, the algorithm may work for majority of inputs but crash on some “boundary” value.

Algorithm design & analysis process



Unit I INTRODUCTION

Ascertaining the capabilities of a computational device:

- The next step is to ascertain the capabilities of the computational device the algorithm is intended for.
- The algorithms in use today are still designed to be programmed for a computer closely resembles the current architecture.
- This architecture is captured by the so-called random-access machine (RAM).
- Its central assumption is that instructions are executed one after another, one operation at a time.
- Accordingly, algorithms designed to be executed on such machines are called sequential algorithms.
- The central assumptions does not hold for some newer computers that can execute operations concurrently. i.e in parallel. This type of algorithms are called parallel algorithms.
- If it is a scientific problem, the algorithm can be designed independent of specification parameters for a particular computer.
- If we were designing as a practical tool, the answer may depend on a problem you need to solve.
- In many problems, we are not bothering about a computer speed and memory.
- There are important problems, which are very complex, have to process huge volumes of data, in that case it is imperative to be aware of the speed and memory available on a particular system.

Choosing between exact and approximate problems solving:

- The next principal decision is to choose between solving the problem exactly or solving it approximately.

Unit I INTRODUCTION

- In former case, it is solving the problem exactly called exact algorithm, latter approximation algorithm.
- What is the need for approximation algorithm?
 - i) There are important problems that cannot be solved exactly such as
 - Extracting square roots.
 - Solving non-linear equations.
 - Evaluation definite integrals.
 - ii) The algorithms for solving a problem exactly can be unacceptably slow because of the problems intrinsic complexity.
 - Ex: Travelling salesman problem.
 - iii) Approximation algorithm can be part of a more sophisticated algorithm that solves a problem exactly.

Deciding on approximate data structures:

- Some algorithms do not demand any ingenuity in representing their inputs.
- But others are, in fact predicated on ingenious data structures.

Algorithm design techniques:

- An algorithm design technique is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.
- Need for learning algorithm design techniques:
 - i) They provide guidance for designing algorithms for new problems.
 - ii) Algorithm design techniques make it possible to classify algorithms according to an underlying design idea, therefore, they can serve as natural way to both categorize and study algorithm.

Unit I INTRODUCTION

• Methods of specifying an algorithm:

- There are two options that are most widely used nowadays for specifying algorithms:
 - i) Pseudo code
 - ii) Flowchart
- Pseudo code is a mixture of a natural language and programming language like constructs.
- A pseudo code is usually more precise than a natural language and its usage of ten yields more algorithm description.
- Flowchart a method of expressing algorithm by a collection of connected geometric shapes containing descriptions of the algorithms steps.
- Pseudo code cannot be directly fed into a computer, instead it needs to be converted into a computer program written in a particular computer language.

• Proving an algorithms correctness:

- Once an algorithm has been specified, you have to prove its correctness.
- That it, you have to prove that the algorithm yields a required result for every legitimate input in a finite amount of time.

For Ex: Euclid's algorithm for computing the gcd

$$\text{gcd}(m,n) = \text{gcd}(n,m \bmod n)$$

- The simple observation that the second number gets smaller on every iteration of the algorithm.
- Algorithm gets terminated when $m \bmod n$ becomes 0.
- For some algorithms, a proof of correctness is quite easy; for others, it can be quite complex.
- A common technique for proving correctness is to use mathematical induction.

Unit I INTRODUCTION

- In order to show that an algorithm is incorrect, we need just one instance of its input of which the algorithm fails.
- If the algorithm is found to be incorrect, we need to redesign it.

Analyzing an algorithm:

- After correctness, the most important quality is efficiency.
- There are two kinds of algorithm efficiency
 - i) Time Efficiency.
 - ii) Space Efficiency.
- Time Efficiency indicates how fast the algorithm runs.
- Space Efficiency indicates how much extra memory the algorithm needs.
- Another desirable characteristic of an algorithm is simplicity.
- Simpler algorithms are easier to understand and easier to program. The resulting programs usually contain fewer bugs.
- Another desirable characteristic of an algorithm is generality.
- There are two issues:
 - i) Generality of the problem the algorithm solves.
 - ii) The range of inputs it accepts.

First issue:

- Sometimes easier to design an algorithm for a problem posed in more general terms.
Ex: gcd
- There are situations where designing a more general algorithm is unnecessary or difficult or even impossible.

Ex: Quadratic equation cannot be generalized.

Unit I INTRODUCTION

Second issue:

- Designing an algorithm that can handle a range of inputs that is natural for the problem at hand.
- If we are not satisfied with the algorithms efficiency, simplicity, generality, we must redesign the algorithm.
- In fact, even evaluation is positive, it is still worth searching for other algorithms solutions.

Coding an algorithm:

- Most algorithms are destined to be ultimately implemented as computer programs.
- Correctness of the computer program is proven with mathematical rigor.
- As a practical matter, the validity of programs is still established by testing.
- Test and debug the program thoroughly whenever we implement an algorithm.
- Implementing an algorithm correctly is necessary but not sufficient. We should have efficient algorithm.
- Better algorithm can make a difference in running time by orders of magnitude.
- Empirical analysis of the algorithm is based on timing the program on several inputs and then analyzing the results obtained.
- An important issue of algorithms problem solving is whether or not every problem can be solved by an algorithm.
- Ex: Finding real root of a quadratic equation with a negative discriminant there is no solution.

Unit I INTRODUCTION

1.3. Fundamentals of the analysis of algorithm efficiency:

- The term “analysis of algorithms” mean an investigation of an algorithms efficiency with respect to two resources: running time and memory space.

Analysis framework:

- Outlining a general framework for analyzing the efficiency of algorithms.
- There are two kinds of efficiency:
 - i) **Time efficiency** indicates how fast an algorithm runs.
 - ii) **Space efficiency** deals with extra space the algorithm requires.

Measuring an input's size:

- The most obvious observation that almost all algorithms run longer on larger inputs.
Ex: It takes longer to sort larger arrays, multiply larger matrices and so on.
- Therefore, it is logical to investigate an algorithm's efficiency as a function of some parameter n indicating the algorithm's input size.
- In some cases, selecting such a parameter is quite straight forward.

Ex: Sorting, searching $\square$ where the size of the list will be taken.

- For the problem of evaluating a polynomial

$$p(x) = a_n x_n + + a_0 \text{ of degree } n.$$

Parameters may be

- i) Polynomial's degree
- ii) Number of its co-efficient.

Unit I INTRODUCTION

In some situation, where the input size matters.

Ex: Computing the product of two $n \times n$ matrices.

There are two natural measures:

- i) Matrix order n
 - ii) Total number of element N .
- So, the algorithm efficiency will be qualitatively different depending on which of the two measures we use.
 - The choice of an appropriate size metric can be influenced by operations of the algorithm in question.

Ex: Measuring input's size for spell checking algorithm.

- i) If the algorithm examines individual characters of its input, then measure the size by the number of characters.
 - ii) If it works by processing words, should count their number in the input.
- Should make a special note about measuring size of inputs for algorithms involving properties of numbers.

Ex: Checking for prime.

Computer scientists prefer measuring size by the number of bits b in the n 's binary representation.

$$b = \lceil \log_2 n \rceil + 1$$

This gives the efficiency of algorithms.

Unit I INTRODUCTION

Units for measuring running time:

- Concerns about units of measuring an algorithm's running time.
- We can simply use some standard unit of time measurement – a second, a millisecond and so on to measure the running time of a program implementing the algorithm.

Drawbacks:

- i) Dependence on the speed of a particular computer.
- ii) Dependence on the quality of a program implementing the algorithm.
- iii) The compiler used in generating the machine code.
- iv) The difficulty of clocking the actual running time of the program.
- We should have a metric that does not depend on these extraneous factors.
- One possible approach is to identify the most important operation of the algorithm, called the basic operation. The operation contributing the most for the total running time and compute the number of times the basic operation is executed.
- It is not difficult to identify the basic operation of an algorithm.

Ex-1: Sorting algorithms work by comparing elements of a list being sorted with each other. The basic operation is a key comparison.

Ex-2 : Matrix multiplication and polynomial evaluation which requires two arithmetic operations: Multiplication and addition.

- Thus, the established frame work for the analysis of an algorithm's time efficiency suggests measuring it by counting the number of time the algorithm's basic operation is executed in inputs of size n .
- The important application

C_{op} – The time of execution of an algorithm's basic operation on a particular computer.

$C(n)$ – The number of times this operation needs to be executed for this algorithm.

Unit I INTRODUCTION

Estimating the running time $T(n)$ of a program implementing this algorithm on that computer is

$$T(n) \approx C_{op} C(n)$$

- In this formula, the count $C(n)$ does not contain any information about operations that are not basic. So, the count is computed approximately.
- C_{op} is also approximation.
- Unless n is extremely small or large, the formula can give a reasonable estimate of the algorithm's running time.

Assuming that $C(n) = \frac{1}{2} n (n-1)$

How much longer will the algorithm run if double its input size?

$$C(n) = \frac{1}{2} n (n-1) = \frac{1}{2} n^2 - \frac{1}{2} n \approx \frac{1}{2} n^2$$

and therefore

$$\frac{T(2n)}{T(n)} \approx \frac{c_{op} C(2n)}{c_{op} C(n)} \approx \frac{\frac{1}{2} (2n)^2}{\frac{1}{2} n^2} = 4.$$

The answer is 4 times longer

- C_{op} value is not knowing. The value was neatly cancelled out in the ratio.
- $\frac{1}{2}$ the multiplicative constant in the formula for the count $C(n)$, was also cancelled out.

Therefore, the efficiency analysis framework ignores multiplicative constants and concentrates on the count's order of growth to within a constant multiple for larger-size inputs.

Unit I INTRODUCTION

Orders of growth:

- A difference in running times on small inputs is not what really distinguishes efficient algorithms from inefficient ones.

Ex: Computing gcd of two small numbers, the efficiency is not clear when Euclid's algorithm is compared with other two algorithms.

It is only clear when we find gcd of two large numbers.

- The magnitude of the numbers in Table 1 has a profound significance for the analysis of algorithm.
- The function growing the slowest among these is the logarithmic function.

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		

Table 1 Values (some approximate) of several functions important for analysis of algorithms

- On the other hand of the spectrum are the exponential function 2^n and $n!$. Both these functions grow so fast that their values become astronomically large even for rather small values of n .
- Another way to appreciate the qualitative difference among the orders of growth of the functions in table is if we increase two fold in the value of their argument n .

Unit I INTRODUCTION

The function

$$\log_2 n \Rightarrow \log_2 2n = \log_2 2 + \log_2 n = 1 + \log_2 n \Rightarrow \text{increases in value by just 1.}$$

$$n \log_2 n \Rightarrow 2n \log_2 2n = 2n [1 + \log_2 n] \Rightarrow \text{increases slightly more than 2 fold.}$$

$$n^2 \Rightarrow (2n)^2 = 4n^2 \Rightarrow 4 \text{ fold}$$

$$n^3 \Rightarrow (2n)^3 = 8n^3 \Rightarrow 8 \text{ fold}$$

$n!$ = increases much more than the above

Worst-case, Best-case and Average-case Efficiencies:

- To measure an algorithm's efficiency as a function of a parameter indicating the size of the algorithm's input.
- But there are many algorithms for which running time depends not only on an input size but also on the specific of a particular input.

Ex: Sequential search.

This is a straight forward algorithm that searches for a given item in a list of n elements by checking successive elements of the list until either a match with the search key is found or the list is exhausted.

Algorithm:

• Sequential search ($A[0 \dots n-1]$, K)

// Input: An array $A[0 \dots n-1]$ and a search key K .

// Output: Returns the index of the first element of A that matches K or -1 if there are no matching elements.

$i \leftarrow 0$

while $i < n$ and $A[i] \neq K$ do

$i \leftarrow i + 1$

if $i < n$ return i

else return -1

Unit I INTRODUCTION

- The running time of this algorithm can be quite different for the same list size n depends where the K appears.
- In the worst case, when there are no matching elements or the first matching element happens to be the last one on the list, the algorithm makes the largest number of key comparisons among all possible inputs of size n :

$$C_{\text{worst}}(n) = n$$

- The **Worst Case Efficiency (WCE)** of an algorithm is its efficiency for the worst-case input of size n , which is an input of size n for which the algorithm runs the longest among all possible inputs of that size.
- Computing worst-case efficiency is straight forward, analyze what kind of inputs yield the largest value of the basic operation's count $C(n)$ among all possible inputs of size n and then compute the worst-case value **$C_{\text{worst}}(n)$**
- WCE provides important information about an algorithm's efficiency by bounding its running time.

It guarantees the running time will not exceed $C_{\text{worst}}(n)$

- The **Best Case Efficiency (BCE)** of an algorithm is its efficiency for the best-case input of size (n) , which is an input of size n for which the algorithm runs fastest among all possible inputs of that size.

Analyze the BCE:

- i) Determine the kind of inputs for which the count $C(n)$ will be the smallest among all possible inputs of size n .
- ii) Ascertain the value of $C(n)$ on the most convenient inputs.

Ex: Sequential search

The first element is equal to a search key.

$$C_{\text{best}}(n) = 1$$

Unit I INTRODUCTION

Average Case Efficiency provide the necessary information about an algorithm's behavior on a "typical" or "random" input.

- We must take some assumptions about possible inputs of size n .
- The standard assumptions are

i) The probability of a successful search is equal to p ($0 \leq p \leq 1$)

ii) The probability of the first match occurring in the i^{th} position of the list is the same for every i .

- Find the average number of key comparisons.

$C_{\text{avg}}(n)$ as follows

Successful Search

- The probability of the first match occurring in the i^{th} position of the list is p/n for every i .
- The number of comparisons in such a situation is obviously i .

Unsuccessful search

- The number of comparison is n
- Probability is $(1-p)$

$$\begin{aligned} C_{\text{avg}}(n) &= \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \dots + i \cdot \frac{p}{n} + \dots + n \cdot \frac{p}{n} \right] + n \cdot (1-p) \\ &= \frac{p}{n} [1 + 2 + \dots + i + \dots + n] + n(1-p) \\ &= \frac{p}{n} \frac{n(n+1)}{2} + n(1-p) = \frac{p(n+1)}{2} + n(1-p). \end{aligned}$$

Unit I INTRODUCTION

if p = 1

The average number of key comparisons = $(n+1)/2 + n(1-1)$
 $= (n+1)/2$

(i.e) The algorithm will inspect, on average, about half of the list's elements.

if p = 0

The average number of key comparisons = $0(n+1)/2 + n(1-0)$
 $= n$

Therefore, the algorithm will inspect all n elements on all such inputs.

- Investigation of the ACE is considerably more difficult than investigation of the worst-case & best-case efficiencies.
- The direct approach for doing it involves dividing all instances of size n into several classes so that for each instance of the class the number of times the algorithm's basic operation is executed is the same.
- The probability distribution of inputs need to be obtained or assumed so that the expected value of the basic operation's count then be derived.

Amortized Efficiency:

- It applies not to a single run of an algorithm but rather to a sequence of operations performed on the same data structure.
- It turns out that in some situations a single operation can be expensive, but the total time for an entire sequence of n such operations is always significantly better than the worst-case efficiency of that single operation multiplied by n.

Unit I INTRODUCTION

1.4 Asymptotic Notations And Basic Efficiency Classes:

- To compare orders of growth, there are three notations

i) O (Big Oh)

ii) Ω (Big Omega)

iii) θ (Big Theta)

$t(n)$ and $g(n)$ can be any non-negative functions defined on the set of natural numbers.

$t(n)$ = algorithms running time.

$g(n)$ = simple function to compare the count.

Introduction

- $\Omega(g(n))$ stands for the set of all function with a larger or same order of growth as $g(n)$.

$$n^3 \in \Omega(n^2), \quad \frac{1}{2} n(n-1) \in \Omega(n^2),$$

$$100 + 5 \notin \Omega(n^2)$$

- $\theta(g(n))$ is a set of all functions that have the same growth as $g(n)$.

$$an^2 + bn + c \text{ with } a > 0 \quad \theta(n^2)$$

Asymptotic Notations and its Properties

Informal Introduction

$O(g(n))$ is the set of all functions with a smaller or same order of growth as $g(n)$.

Examples:

$$n \in O(n^2), \quad 100n+5 \in O(n^2), \quad \frac{1}{2}n(n-1) \in O(n^2),$$

The first two function are linear and have a smaller order of growth than $g(n)=n^2$. But the last one quadratic and has the same order of growth as n^2 .

Unit I INTRODUCTION

The second notation $\Omega(g(n))$, set of all functions with a larger or same order of growth as $g(n)$.

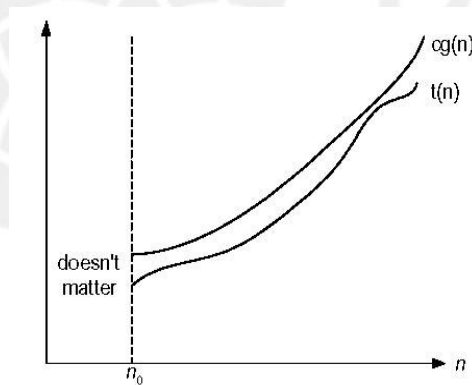
Examples:

$$n^3 \in \Omega(n^2), \quad \frac{1}{2}n(n-1) \in \Omega(n^2)$$

Finally $\Theta(g(n))$, set of all functions have the same order of growth as $g(n)$. Thus every quadratic equation, an^2+bn+c with $a>0$ is in $\Theta(n^2)$.

O-notation

A function $t(n)$ is said to be in $O(g(n))$, denoted $t(n) \in O(g(n))$, if $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and non-negative integer n_0 such that $t(n) \leq cg(n)$ for all $n \geq n_0$



Example:

$$100n+5 \in O(n^2)$$

$$100n+5 \leq 100n+n \text{ (for all } n \geq 5) = 101n \leq 101n^2$$

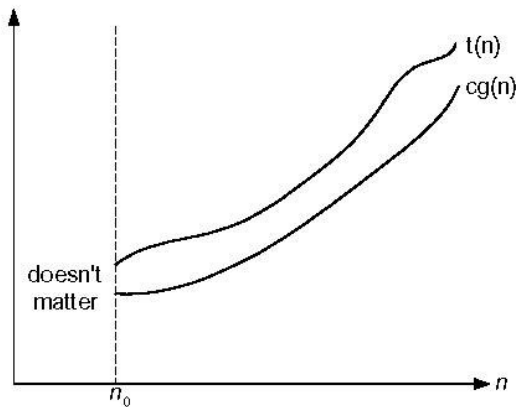
As the values of constants c and n_0 required by the definition, we take 101 and 5 respectively.

$$100n+5 \leq 100n+5n \text{ (for all } n \geq 1) = 105n \text{ with } c=105 \text{ and } n_0=1$$

Unit I INTRODUCTION

Ω -notation

A function $t(n)$ is said to be in $\Omega(g(n))$, denoted $t(n) \in \Omega(g(n))$, if $t(n)$ is bounded below by some positive constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and some nonnegative integer n_0 such that $t(n) \geq cg(n)$ for all $n \geq n_0$.



Example:

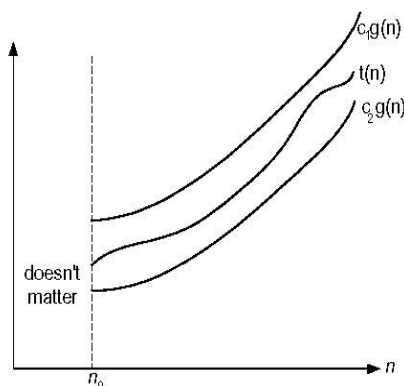
$$n^3 \in \Omega(n^2)$$

$$n^3 \geq n^2 \text{ for all } n \geq 0$$

i.e., we can select $c=1$ and $n_0=0$.

Θ -notation

A function $t(n)$ is said to be in $\Theta(g(n))$, denoted $t(n) \in \Theta(g(n))$, if $t(n)$ is bounded both above and below by some positive constant multiples of $g(n)$ for all large n , i.e., if there exist some positive constant c_1 and c_2 and some nonnegative integer n_0 such that $c_2g(n) \leq t(n) \leq c_1g(n)$ for all $n \geq n_0$.



Unit I INTRODUCTION

Prove

Left inequality,

Right inequality,

Hence we select $c_2 = 1/4$, $c_1 = 1/2$ and $n_0 = 2$

$$\frac{1}{2}n(n-1) \in \theta(n^2)$$

$$\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2 \quad \text{for all } n_0 \geq 1$$

$$\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{2}n^2 - \frac{1}{2}n \cdot \frac{1}{2}n = \frac{1}{4}n^2 \quad \text{for all } n \geq 2$$

Properties of Asymptotic Notations

Theorem: If $t_1(n) \in O(g_1(n))$ and $t_2 \in O(g_2(n))$ then $t_1(n)+t_2(n) \in O(\max\{g_1(n),g_2(n)\})$

Proof: simple fact about four arbitrary real numbers a_1, a_2, b_1, b_2 : if $a_1 \leq b_1$ and $a_2 \leq b_2$, then $a_1+a_2 \leq 2 \max\{b_1,b_2\}$.

Since if $t_1(n) \in O(g_1(n))$, there exist some positive constant c_1 and some nonnegative integer n_1 such that

$$t_1(n) \leq c_1 g_1(n) \quad \text{for all } n \geq n_1$$

Since if $t_2(n) \in O(g_2(n))$, $t_2(n) \leq c_2 g_2(n)$ for all $n \geq n_2$

Let us denote $c_3 = \max\{c_1, c_2\}$ and consider $n \geq \max\{n_1, n_2\}$ we can use both inequalities.

$$\begin{aligned} T_1(n) + t_2(n) &\leq c_1 g_1(n) + c_2 g_2(n) \\ &\leq c_3 g_1(n) + c_3 g_2(n) = c_3 [g_1(n) + g_2(n)] \\ &\leq c_3 2 \max\{g_1(n), g_2(n)\} \end{aligned}$$

So $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$

It implies that the algorithm's overall efficiency is determined by the part with a larger order of growth.

Unit I INTRODUCTION

Comparing Order's of Growth using Limits.

- Though definitions of O , Ω and θ are indispensable for proving their abstract properties, they are rarely used for comparisons of orders of growth of two specific functions.
- A much more convenient method for doing so is based on computing the limit of the ratio of two functions.

Three principal cases may arise

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} \begin{cases} 0 & \text{implies that } t(n) \text{ has a smaller order of growth than } g(n) \\ \neq 0 & \text{implies that } t(n) \text{ has a same order of growth as } g(n) \\ \infty & \text{implies that } t(n) \text{ has a larger order of growth than } g(n) \end{cases}$$

First two cases mean that $t(n) \in O(g(n))$

Last two cases mean that $t(n) \in \Omega(g(n))$

Second case mean that $t(n) \in \theta(g(n))$

L'Hospital's rule

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{t'(n)}{g'(n)}$$

Stirling's formula

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \text{ for large values of } n$$

Example 1: compare the order's of growth of $\frac{1}{2} n(n-1)$ and n^2

$$\lim_{n \rightarrow \infty} \frac{\frac{1}{2} n(n-1)}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \frac{n^2 - n}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right) = \frac{1}{2}$$

Since it is equal to a +ive constant, the functions have the same order of growth. So

$$\frac{1}{2} n(n-1) \in \theta(n^2)$$

Unit I INTRODUCTION

Example 2: compare the order's of growth of $\log_2 n$ and $\sqrt{n}$

$$\lim_{n \rightarrow \infty} \frac{\log_2 n}{\sqrt{n}} = \lim_{n \rightarrow \infty} \frac{(\log_2 n)'}{(\sqrt{n})'} = \lim_{n \rightarrow \infty} \frac{\log_2 e (\frac{1}{n})}{\frac{1}{2\sqrt{n}}} = 2 \log_2 e \lim_{n \rightarrow \infty} \frac{\sqrt{n}}{n} = 0$$

Since it is 0, $\log_2 n$ has smaller order of growth than $\sqrt{n}$. So $\log_2 n \in O(\sqrt{n})$

Example 3: compare the order's of growth of $n!$ and 2^n

$$\lim_{n \rightarrow \infty} \frac{n!}{2^n} = \lim_{n \rightarrow \infty} \frac{\sqrt{2\pi n} (\frac{n}{e})^n}{2^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} \frac{n^n}{2^n e^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} (\frac{n}{2e})^n = \infty$$

2^n grows very fast. $N!$ still grows faster. We can write $n! \in (2^n)$.

Basic Efficiency Classes

- The time efficiencies of a large number of algorithms fall into only a few classes.
- These classes are listed in increasing order of their growth along with their names and a few comments.

Basic Efficiency Classes

Order of Growth	Description
1	Short of <u>best case</u> efficiencies algorithm's running time typically goes to infinity when its input size grows infinitely large
$\log n$	Result of cutting a problem's size by a constant factor on each iteration of the algorithm
n	Algorithm's that scan a list of size n belongs to this class
$n \log n$	Divide and conquer algorithms like merge sort, quick sort etc.
n^2	Characterizes efficiency of algorithms with two embedded loops. Elementary sorting algorithms and matrix addition
n^3	Algorithms embedded with three loops
2^n	Algorithms that generate all subsets of an n -element set
$n!$	Algorithms that generate all permutations of an n -element set

Unit I INTRODUCTION

1.5. Mathematical Analysis of Non-Recursive & Recursive Algorithms:

1.5.1 Mathematical Analysis of Non-Recursive Algorithms

- Analyzing the efficiency of non-recursive algorithms.
- Before proceeding with examples, we review about list of summation formulas and two basic rules of sum manipulation.

$$\sum_{i=l}^u ca_i = c \sum_{i=l}^u a_i, \quad (\text{R1})$$

$$\sum_{i=l}^u (a_i \pm b_i) = \sum_{i=l}^u a_i \pm \sum_{i=l}^u b_i, \quad (\text{R2})$$

$$\sum_{i=l}^u 1 = u - l + 1 \quad \text{where } l \leq u \text{ are some lower and upper integer limits, } (\text{S1})$$

$$\sum_{i=0}^n i = \sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2). \quad (\text{S2})$$

Problem: Finding the value of the largest element in a list of n numbers.

Assumption: List is implemented as array.

Algorithm MaxElement(A[0..n-1])

//To find the largest element

//Input: Array of n elements

//Output: Largest element

maxval $\square$ A[0]

for i $\square$ 1 to n-1 do

 if A[i] > maxval

 maxval $\square$ A[i]

return maxval

Unit I INTRODUCTION

- Applying general framework outlined.

i) Measuring an input's size:

- The input size is the number of elements in the array i.e., n

i) Units for measuring running time:

- The operations most often executed are in the for loop.
- There are two operations
 - Comparison $A[i] > \text{maxval}$
 - Assignment $\text{maxval} \leftarrow A[i]$

Which have to consider?

- Comparison is executed on each repetition of the loop
- Assignment is no. So, the basic operation is comparison
- The number of comparisons will be the same for all arrays of size n . There is no need to distinguish among the worst, average and best cases.
- Count the number of times the basic operation performed and try to formulate as function of size n .
- The algorithm makes one comparison on each execution of the loop, which is repeated for each value of the loop's variable i within the bounds between 1 and $n-1$.

$$C(n) = \sum_{i=1}^{n-1} 1 \Rightarrow \sum_{i=1}^{n-1} 1 = n - 1 - 1 + 1 = n - 1 \in \theta(n)$$

Unit I INTRODUCTION

General plan for analyzing efficiency of non-recursive algorithms:

- i) Decide on a parameter indicating an input's size.
- ii) Identify the algorithm's basic operation.
- iii) Check whether the number of times the basic operation is executed depends only on the size of an input.
- iv) Set up a sum expressing the number of times the algorithm's basic operation is executed.
- v) Establish its order of growth.

Example 2: Element uniqueness problem:

Algorithm UniqueElements (A[0..n-1])

//To check the uniqueness of an array

//Input: An array of n elements

//Output: Returns true if the array does not have duplicate elements. Otherwise false

for i $\square$ 0 to n-2 do

 for j $\square$ i+1 to n-1 do

 if A[i] = A[j] return false

return true

i) Measuring input's size:

- The number of elements in the array- n elements

ii) Measuring running time:

- The innermost loops contains a single operation.
- This is the basic operation
- The number of elements comparison will depend not only on n but also on whether there are equal elements in the array and if there are, which array positions they occupy..

Unit I INTRODUCTION

- We consider worst case only.
- The innermost loop reveals that there are two kinds of worst case inputs.
 - Arrays with no equal elements.
 - Arrays in which the last two elements are the only pair of equal elements.

$$\begin{aligned}C_{worst}(n) &= \sum_{i=0}^{n-2} 1 \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1) - (i+1) + 1 \\&= \sum_{i=0}^{n-2} (n-1-i-1+1) = \sum_{i=0}^{n-2} (n-1-i)\end{aligned}$$

$$\begin{aligned}&= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \sum_{i=0}^{n-2} i \\&= (n-1)(n-2-0+1) - \frac{(n-2)(n-1)}{2} \\&= (n-1)(n-1) - \frac{(n-2)(n-1)}{2} = (n-1)^2 - \frac{(n-2)(n-1)}{2} \\&= (n-1) \left[(n-1) - \frac{1}{2}(n-2) \right] \\&= (n-1) \left[\frac{2n-2-n+2}{2} \right] \\&= (n-1) \left[\frac{n}{2} \right]^2 = \frac{n(n-1)}{2} \approx \frac{1}{2} n^2 \in \theta(n^2)\end{aligned}$$

Therefore, algorithm needs to compare all $n(n-1)/2$ distinct pairs of its n elements.

Unit I INTRODUCTION

Example 3: Given two nxn matrices. Find the Product of two matrices

ALGORITHM MatrixMul(A[0..n-1,0..n-1],B[0..n-1,0..n-1])

// To find the product of two matrices of order nxn

//Input: A and B are two matrices of nxn order

//Output: Resultant matrix C

for i□0 to n-1 do

 for j□0 to n-1 do

 C[i,j]=0

 for k□0 to n-1 do

 C[i,j]=c[i,j]+A[i,k]*B[k,j]

return C

Algorithm analysis

- An input's size is matrix order n.
- There are two arithmetical operations (multiplication and addition) in the innermost loop. But we consider multiplication as the basic operation.
- Let us set up a sum for the total number of multiplications M(n) executed by the algorithm. Since this count depends only on the size of the input matrices, we do not have to investigate the worst-case, average-case, and best-case efficiencies separately.
- There is just one multiplication executed on each repetition of the algorithm's innermost loop, which is governed by the variable k ranging from the lower bound 0 to the upper bound n – 1.
- Therefore, the number of multiplications made for every pair of specific values of variables i and j is

$$\sum_{k=0}^{n-1} 1$$

The total number of multiplications M(n) is expressed by the following triple sum:

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

The sum is computed by using the formula (S1) and Rule (R1)

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3$$

Unit I INTRODUCTION

Example 4: Finds the number of digits in the binary representation of a positive decimal integer

ALGORITHM Binary(n)

//To find the count of digits in a binary representation of a number

//Input: The number n

//Output: number of digits

count $\leftarrow$ 1

while $n > 1$ do

 count $\leftarrow$ count + 1

$n \leftarrow \lfloor n/2 \rfloor$

Algorithm analysis:

- An input's size is n.
 - The loop variable takes on only a few values between its lower and upper limits.
 - Since the value of n is about halved on each repetition of the loop, the answer should be about $\log_2 n$.
 - The exact formula for the number of times the comparison $n > 1$ will be executed is actually $\log_2 n + 1$.
-
- The number of times, the comparison will be executed is larger than the number of repetitions of the loop's body by exactly 1. The choice is not that important.
 - The value of n is about halved on each repetition of the loop. So it is $\log_2 n$.
 - Therefore, the exact formula for the number of times the comparison $n > 1$ will be executed is $\log_2 n + 1$
 - So alternative computing loop's variable takes on only a few values between its upper & lower limits.

Unit I INTRODUCTION

Mathematical Analysis of recursive algorithms

- The general frame work for analysis of algorithms to analyzing recursive algorithms.

Example 1: Factorial of a number F(n)

if $n = 0$ return 1

else return $F(n-1) * n$

i) Measuring input's size:

- n is the input's size

i) Measuring running time:

- The basic operation of the algorithm is multiplication, whose number of executions we denote $M(n)$. Since the function $F(n)$ is computed according to the formula

$$F(n) = F(n - 1) \cdot n \text{ for } n > 0$$

$$M(n) = M(n - 1) + 1 \text{ for } n > 0$$

This equation is called recurrence relation

- To determine a solution uniquely for recurrence relation, we need an initial condition that tells us the value with which the sequence starts.
- We can obtain this value by inspecting the condition that makes the algorithm stop its recursive calls.

if $n = 0$ return 1

$M(0) = 0$

The call stop when $n = 0$

- There are two recursively defined functions
 - Factorial function $F(n)$
 - Number of multiplications $M(n)$

Unit I INTRODUCTION

- For solving recurrence relations, we use the method of backward substitutions

$$M(n) = M(n-1) + 1 \quad \text{substitute } M(n-1) = M(n-2) + 1$$

$$= [M(n-2) + 1] + 1 = M(n-2) + 2 \quad \text{substitute } M(n-2) = M(n-3) + 1$$

$$= [M(n-3) + 1] + 2 = M(n-3) + 3$$

$$\text{Therefore, general formula} = M(n) = M(n-i) + i$$

$$M(n) = M(n-1) + 1 \dots = M(n-i) + i = \dots$$

Initial condition $n=0$ so substitute $i=n$

$$M(n-n) + n = n$$

General plan for analyzing efficiency of recursive algorithms:

- 1) Decide on a parameter.
- 2) Identify the algorithm's basic operation.
- 3) Measuring running time.
- 4) Setup a recurrence relation, with an appropriate initial condition for the number of times the basic operation is executed.
- 5) Solve the recurrence.

Example 2: Towers of Hanoi problem

1. First move recursively $(n-1)$ disks from peg 1 to peg2, peg 3 as auxiliary
 2. Move the largest disk directly from peg 1 to peg3.
 3. Move recursively $(n-1)$ disks from peg 2 to peg3, peg 1 as auxiliary
- i) Measuring input size:
- The number of disks n is the input size indicator.

Unit I INTRODUCTION

(ii) Measuring running time:

- The algorithm's basic operation is moving one disk.
- Clearly, the number of moves $M(n)$ depends on n only.
- So, the following recurrence equation for it

$$\begin{aligned}M(n) &= M(n-1) + 1 + M(n-1) \quad \text{for } n > 1 \\&= M(n-1) + 1 + M(n-1)\end{aligned}$$

- Obvious initial condition $M(1) = 1$

$$\begin{aligned}M(n) &= 2M(n-1) + 1 \quad \text{for } n > 1 \\M(1) &= 1\end{aligned}$$

Solving recurrence relation equation by backward substitutions

$$\begin{aligned}M(n) &= 2M(n-1) + 1 & \text{sub } M(n-1) &= 2M(n-2) + 1 \\&= 2 [2M(n-2) + 1] + 1 \\&= 2^2 M(n-2) + 2 + 1 & \text{sub } M(n-2) &= 2M(n-3) + 1 \\&= 2^2 [2M(n-3) + 1] + 2 + 1 \\&= 2^3 M(n-3) + 2^2 + 2 + 1\end{aligned}$$

Therefore, after i substitutions, we get

$$\begin{aligned}M(n) &= 2^i M(n-i) + 2^{i-1} + 2^{i-2} + \dots + 2 + 1 \\&= 2^i M(n-i) + 2^i - 1 \\M(n) &= 2^{n-1} M(n-(n-1)) + 2^{n-1} - 1 & n-i &= 1, i = n-1 \\&= 2^{n-1} M(1) + 2^{n-1} - 1 \\&= 2^{n-1} + 2^{n-1} - 1 \\&= 2 \cdot 2^{n-1} - 1 = 2^n - 1\end{aligned}$$

Unit I INTRODUCTION

Thus we have an exponential algorithm, which will run for an unimaginably long time even for moderate values of n . This is not due to the fact that this algorithm is poor.

It is the problem's intrinsic difficulty that makes it so computationally difficult.

Example 3: Counting the number of binary digits

Algorithm: Bin Rec(n)

if $n = 1$ return 1

else return Bin Rec($\lceil n/2 \rceil$) + 1

- For $n = 2^k$ takes the form

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{for } k > 0$$

$$A(2^0) = 0$$

Now backward substitutions encounter no problems:

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{substitute } A(2^{k-1}) = A(2^{k-2}) + 1$$

$$= [A(2^{k-2}) + 1] + 1$$

$$= A(2^{k-2}) + 2 \quad \text{substitute } A(2^{k-2}) = A(2^{k-3}) + 1$$

$$= [A(2^{k-3}) + 1] + 2$$

$$= A[2^{k-3}] + 3$$

$$= A[2^{k-i}] + i$$

$$= A[2^{k-k}] + k$$

Unit I INTRODUCTION

- For $n = 2^k$ takes the form

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{for } k > 0$$

$$A(2^0) = 0$$

Now backward substitutions encounter no problems:

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{substitute } A(2^{k-1}) = A(2^{k-2}) + 1$$

$$= [A(2^{k-2}) + 1] + 1$$

$$= A(2^{k-2}) + 2 \quad \text{substitute } A(2^{k-2}) = A(2^{k-3}) + 1$$

$$= [A(2^{k-3}) + 1] + 2$$

$$= A[2^{k-3}] + 3$$

$$= A[2^{k-i}] + i$$

$$= A[2^{k-k}] + k$$

Thus we end up with

$$A(2^k) = A(1) + K$$

$$= 0 + k = k$$

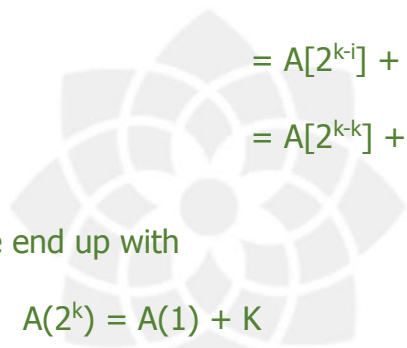
After returning to the original variable $n = 2^k$

Taking log on both sides.

Hence, $\log n = k \log 2$

$$K = \log_2 n$$

$$A(n) = \log_2 n \in \theta(\log n)$$



R.M.K.
GROUP OF
INSTITUTIONS

Unit I INTRODUCTION

Example 4 – Fibonacci Numbers

Fibonacci Series

0, 1, 1, 2, 3, 5

- We can define by simple recurrence

$$F(n) = F(n-1) + F(n-2) \quad \text{for } n > 1$$

And two initial conditions

$$F(0) = 0, F(1) = 1$$

Explicit formula for the n^{th} Fibonacci number

- Backward substitutions will fail to get an easily discernible pattern.
- So, we take a theorem that describes solutions to a homogeneous second-order linear recurrence with constant co-efficients.

$$ax(n) + bx(n-1) + cx(n-2) = 0$$

a, b, c are coefficients of recurrence.

$x(n)$ □ unknown sequence to be found



R.M.K.
GROUP OF
INSTITUTIONS

Assignments

Assignments

1. You are given an array of n positive integers. Design an **algorithmic strategy** (not code) to determine whether there exist two elements whose sum is exactly K . **(CO1, K4)**

Tasks:

- Provide the step-by-step algorithm using algorithmic design principles.
- Analyze the **time and space complexity**.
- Suggest how the solution changes if the array is already **sorted**.
- Compare the efficiencies of both approaches using **asymptotic notation**.

2. Consider the following functions: **(CO1, K4)**

- $f_1(n) = 2n + 10$
- $f_2(n) = 5n \log n$
- $f_3(n) = n^2 / 2$
- $f_4(n) = 3^{2n}$
- $f_5(n) = \sqrt{n} \log n$

Tasks:

- Arrange the functions in **increasing order of asymptotic growth**.
- Provide justification using **formal asymptotic definitions** (Big-O, Big-Ω, Big-Θ).
- Identify which function becomes dominating as $n \rightarrow \infty$.
- Briefly explain why asymptotic notation is used instead of exact running time.

3. For the following recursive algorithm: **(CO1, K4)**



R.M.K.
GROUP OF
INSTITUTIONS

Algorithm Mystery(n):

```
if n <= 1:
    return 1
else:
    return 2 * Mystery(n/2) + n
```

Tasks:

- Derive the recurrence relation.
- Solve the recurrence using **substitution or Master Theorem**.
- Provide the final time complexity in **Big-Theta notation**.
- Explain the contribution of the **non-recursive part** of the algorithm.

4. Consider the following pseudo-code: **(CO1, K4)**

Algorithm Compute(n):

```
sum ← 0
for i = 1 to n:
    for j = 1 to i:
        sum ← sum + 1
return sum
```

Tasks:

- Write a mathematical expression representing the number of basic operations.
- Derive the exact count using the formula $\sum_{i=1}^n i$
- Express the running time using **asymptotic notation** (O , Ω , Θ).
- Compare this with an equivalent **recursive version** if implemented.

5. An algorithm performs the following operations for an input size n : (CO1, K4)

- Step A: $4n^2$ operations
- Step B: $50n \log n$ operations
- Step C: 7×2^n operations

Tasks:

- Identify the **dominant term**.
- Compute the **overall time complexity**.
- Explain how the **analysis framework** helps in breaking down and evaluating algorithmic efficiency.
- If Step C is replaced with a single constant-time operation, re-compute the overall complexity and justify the change.





R.M.K.
GROUP OF
INSTITUTIONS

Part A – Q & A

Unit - I

PART - A Questions

1. What is an Algorithm? (CO1,K1)

An **algorithm** is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.

2. List the sequence of steps in designing and analyzing an algorithm. (CO1,K1)

1. Understand the problem
2. Decide on: computational means, exact vs. approximate solving, algorithm design technique
3. Design an algorithm
4. Prove correctness
5. Analyze the algorithm
6. Code the algorithm

3. What is instance of the problem? (CO1,K1)

An input to an algorithm specifies an **instance** of the problem the algorithm solves. It is very important to specify exactly the set of instances the algorithm needs to handle.

4. Differentiate sequential and parallel algorithm. (CO1,K1)

The essence of design and analyzing architecture is captured by the so-called **random-access machine (RAM)**. Its central assumption is that instructions are executed one after another, one operation at a time. Accordingly, algorithms designed to be executed on such machines are called **sequential algorithms**. The central assumption of the RAM model does not hold for some newer computers that can execute operations concurrently, i.e., in parallel. Algorithms that take advantage of this capability are called **parallel algorithms**.

PART - A Questions

5. Differentiate Exact and Approximate algorithm (CO1,K1)

The principal decision is to choose between solving the problem exactly or solving it approximately. In the former case, an algorithm is called an **exact algorithm** ; in the latter case, an algorithm is called an **approximation algorithm** .

6. What is an algorithm design technique? (CO1,K1)

An **algorithm design technique** (or “strategy” or “paradigm”) is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

7. What is the notion of correctness for algorithms? (CO1,K1)

The notion of correctness for approximation algorithms is less straightforward than it is for exact algorithms. For an approximation algorithm, we usually would like to be able to show that the error produced by the algorithm does not exceed a predefined limit.

8. What are the two kinds of algorithm efficiency: (CO1,K1)

There are two kinds of algorithm efficiency: **time efficiency** , indicating how fast the algorithm runs, and **space efficiency** , indicating how much extra memory it uses.

9. What is basic operation? (CO1,K1)

The thing to do is to identify the most important operation of the algorithm, called the **basic operation** , the operation contributing the most to the total running time, and compute the number of times the basic operation is executed.

10. What is running time of a program implementing an algorithm? (CO1,K1)

The running time $T(n)$ of a program implementing an algorithm by the formula

$$T(n) \approx c_{op}C(n).$$

11. What is an order of growth? (CO1,K1)

The efficiency analysis framework ignores multiplicative constants and concentrates on the count's **order of growth** to within a constant multiple for large-size inputs.

12. Define Best-Case Efficiency. (CO1,K1)

Best-case efficiency of an algorithm is its efficiency for the best-case input of size n , which is an input (or inputs) of size n for which the algorithm runs the fastest among all possible inputs of that size.

What is pseudocode?

13. Define A pseudocode. (CO1,K1)

A pseudocode is a mixture of a natural language and programming language constructs to specify an algorithm. A pseudocode is more precise than a natural language and its usage often yields more concise algorithm descriptions.

14. What are the steps involved in the analysis framework? (CO1,K1)

The various steps are as follows □ Measuring the input's size □ Units for measuring running time □ Orders of growth □ Worst case, best case and average case efficiencies

15. What is the basic operation of an algorithm and how is it identified? (CO1,K1)

The most important operation of the algorithm is called the basic operation of the algorithm, the operation that contributes the most to the total running time. □ It can be identified easily because it is usually the most time consuming operation in the algorithms innermost loop.

16. What are exponential growth functions? (CO1,K1)

The functions 2^n and $n!$ are exponential growth functions, because these two functions grow so fast that their values become astronomically large even for rather smaller values of n .

PART - A Questions

17. What are the basic asymptotic efficiency classes? (CO1, K1)

The various basic efficiency classes are

- Constant : 1
- Logarithmic : $\log n$
- Linear : n
- N-log-n : $n \log n$
- Quadratic : n^2
- Cubic : n^3
- Exponential : 2^n
- Factorial : $n!$

18. What is order of growth? (CO1, K1)

Measuring the performance of an algorithm based on the input size n is called order of growth.

19. What is amortized efficiency? (CO1,K1)

It applies not to a single run of an algorithm but rather to a sequence of operations performed on the same data structure.

20. What is time and space efficiency? (CO1,K1)

time efficiency , indicating how fast the algorithm runs, and *space efficiency* , indicating how much extra memory it uses.

21. Define Average-Case Efficiency? (CO1,K1)

Average-case efficiency is neither the worst-case analysis nor its best-case counterpart yields the necessary information about an algorithm's behavior on a "typical" or "random" input.

PART - A Questions

22. Define Worst-Case Efficiency? (CO1,K1)

Worst-case efficiency of an algorithm is its efficiency for the worst-case input of size n , which is an input (or inputs) of size n for which the algorithm runs the longest among all possible inputs of that size.

23. Explain the Big O asymptotic notation. (CO1,K1)

A function $t(n)$ is said to be in $O(g(n))$, denoted $t(n) \in O(g(n))$, if $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant and some nonnegative integer n_0 such that $t(n) \leq cg(n)$ for all $n \geq n_0$.

24. Explain the Big Ω asymptotic notation. (CO1,K1)

A function $t(n)$ is said to be in $\Omega(g(n))$, denoted $t(n) \in \Omega(g(n))$, if $t(n)$ is bounded below by some positive constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and some nonnegative integer n_0 such that $t(n) \geq cg(n)$ for all $n \geq n_0$.

25. Explain the Big θ asymptotic notation. (CO1,K1)

A function $t(n)$ is said to be in $\theta(g(n))$, denoted $t(n) \in \theta(g(n))$, if $t(n)$ is bounded both above and below by some positive constant multiples of $g(n)$ for all large n , i.e., if there exist some positive constants c_1 and c_2 and some nonnegative integer n_0 such that $c_2g(n) \leq t(n) \leq c_1g(n)$ for all $n \geq n_0$.

26. Write an algorithm to find the number of binary digits in the binary representation of a positive decimal number. (CO1,K1)

ALGORITHM Binary(n)

```
count ← 1
while  $n > 1$  do
    count ← count + 1
     $n \leftarrow \lfloor n/2 \rfloor$ 
return count
```

PART - A Questions

27. Explain the General Plan for Analyzing the Time Efficiency of Nonrecursive Algorithms. (CO1,K1)

1. Understand the experiment's purpose.
2. Decide on the efficiency metric M to be measured and the measurement unit (an operation count vs. a time unit).
3. Decide on characteristics of the input sample (its range, size, and so on).
4. Prepare a program implementing the algorithm (or algorithms) for the experimentation.
5. Generate a sample of inputs.
6. Run the algorithm (or algorithms) on the sample's inputs and record the data observed.
7. Analyze the data obtained.

28. List down the characteristics of an algorithm (CO1,K1)

- Unambiguous
- Simplicity
- Efficient
- Generality

29. Write the best case, worst case and average case efficiency of linear search (CO1,K1)

Best Case $O(1)$

Worst Case $O(n)$

Average Case $O(n)$

30. Arrange the following efficiency classes in increasing order of growth? $\log n, n^2, n!, n, 2^n$ (CO1,K2)

$\log n < n < n^2 < 2^n < n!$

Part A Questions

31. Give the Euclid's algorithm for computing gcd(m, n) (CO1,K1)

ALGORITHM Euclid_gcd(m, n)

//Computes gcd(m, n) by Euclid's algorithm

//Input: Two nonnegative, not-both-zero integers m and n

//Output: Greatest common divisor of m and n

while n ≠ 0 do

r ← m mod n

m ← n

n ← r

return m

Example: gcd(60, 24) = gcd(24, 12) = gcd(12, 0) = 12.

32. Compare the order of growth $n(n-1)/2$ and n^2 (CO1,K1)

$$\lim_{n \rightarrow \infty} \frac{\frac{1}{2}n(n-1)}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \frac{n^2 - n}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right) = \frac{1}{2}$$

33. Design an algorithm to compute the area and circumference of a circle (CO1,K1)

Algorithm Circle(r)

//Input: r, radius of the circle.

//Output: Area and circumference of the circle.

Area = 3.14 * r * r

Circumference = 2 * 3.14 * r

return Area, Circumference

34. Write down the properties of the asymptotic notations. (CO1,K1)

1. If $t_1(n) \in O(g_1(n))$ and $t_2 \in O(g_2(n))$ then $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$
2. Asymptotic notations are transitive, reflexive and symmetric.

35. How to find the running time of an algorithm? (CO1,K1)

The running time $T(n)$ of a program implementing an algorithm by the formula

$$T(n) \approx c_{op}C(n).$$

36. Find the time and space efficiency of the below algorithm: (CO1,K1)

```
for i = 1 to n  
    print i
```

Time efficiency = $O(n)$

Space efficiency 1

37. What are the three different algorithms used to find the gcd of two numbers? (CO1, K1)

The three algorithms used to find the gcd of two numbers are

- Euclid's algorithm
- Consecutive integer checking algorithm
- Middle school procedure



Part B – Questions

Part-B Questions

Q. No.	Questions	CO Level	K Level
1	Discuss about the sequential steps to be followed while designing and analyzing the algorithm.	CO1	K2
2	Explain the basic asymptotic notations.	CO1	K2
3	Explain with suitable examples the analysis of recursive algorithms.	CO1	K3
4	Explain with suitable examples the analysis of non-recursive algorithms.	CO1	K3
5	Analyze the Linear search algorithm	CO1	K3
6	Write and analyze matrix multiplication algorithm	CO1	K3
7	Write and analyze the algorithm for finding the uniqueness of an array.	CO1	K3
8	Write and analyze the algorithm for tower's of Hanoi.	CO1	K3
9	Write and analyze the algorithm for finding factorial of an integer.	CO1	K3
10	Write and analyze the algorithm for counting the number of digits in the binary representation of an integer.	CO1	K3
11	Explain the basic asymptotic notations and their properties.	CO1	K2
12	Write a recursive algorithm for Fibonacci series and analyze it.	CO1	K3
13	Elaborate in detail about the fundamental steps in designing and analyzing the algorithm.	CO1	K2
14	State the general plan for analyzing any recursive algorithm. Analyze the algorithm for counting the number of digits in the binary representation of an integer.	CO1	K3
15	State the general plan for analyzing a non-recursive algorithm. Also write and analyze matrix multiplication algorithm	CO1	K3



R.M.K.
GROUP OF
INSTITUTIONS

GATE Questions

GATE Questions

- Consider the following recurrence relation: $T(n) = 2T(n-1) + n2^n$ for $n > 0$, $T(0) = 1$. Find $T(n)$. (Gate 2025)
- In a balanced binary search tree with n elements, what is the worst case time complexity of reporting all elements in range $[a, b]$? Assume that the number of reported elements is k .
- Let Q denote a queue containing sixteen numbers and S be an empty stack. $\text{Head}(Q)$ returns the element at the head of the queue Q without removing it from Q . Similarly, $\text{Top}(S)$ returns the elements at the top of S without removing it from S . Consider the algorithm given below.

```
while Q is not Empty do
    if S is Empty OR Top(S) ≤ Head(Q) then
        x := Dequeue(Q);
        Push(S,x);
    else
        x := Pop(S);
        Enqueue(Q,x);
    end
end
```

What is the maximum possible number of iterations of the while loop in the algorithm

- Consider the following functions from positive integers to real numbers:

$$10, \sqrt{n}, n, \log_2 n, \frac{100}{n}$$

The CORRECT arrangement of the above functions in increasing order of asymptotic complexity is:

- Consider the following three functions.

$$f_1 = 10^n, f_2 = n^{\log n}, f_3 = n^{\sqrt{n}}$$

The CORRECT arrangement of the above functions in increasing order of asymptotic complexity is:

- The tightest upper bound that represents the time complexity of inserting an object into a binary search tree of n nodes?

GATE Questions

- Two alternate packages A and B are available for processing a database having 10^k records. Package A requires $0.0001n^2$ time units and package B requires time units to process n records. What is the smallest value of k for which package B will be preferred over A?
- Consider the following segment of C-code:

```
int j, n;  
  
j=1;  
  
while(j <= n)  
  
    j = j * 2;
```

The number of comparisons made in the execution of the loop for any $n > 0$ is:

- Let $T(n)$ be the recurrence relation defined as follows:
$$T(0) = 1$$
$$T(1) = 2, \text{ and}$$
$$T(n) = 5T(n - 1) - 6T(n - 2) \text{ for } n \geq 2$$
- Given an integer array of size N , we want to check if the array is sorted (in either ascending or descending order). An algorithm solves this problem by making a single pass through the array and comparing each element of the array only with its adjacent elements. What is the worst-case time complexity of this algorithm?
- The tightest upper bound that represents the number of swaps required to sort n numbers using selection sort?

Supportive online Certification courses (NPTEL, Swayam, Coursera, Udemy, etc.,)

Supportive Online Certification Courses

Sl. No.	Courses	Platform
1	Design and analysis of algorithms	NPTEL
2	The Design and Analysis of Algorithm	Udemy
3	Algorithms Specialization	Coursera
4	Algorithm Design and Analysis	edX
5	Introduction to algorithms and analysis	NPTEL



R.M.K.
GROUP OF
INSTITUTIONS

Real time Applications in day to day life and to Industry

Real Time Applications

1. Write an algorithm to search for a book (Title of the Book is given as a input) in the library. Analyze the efficiency of the algorithm.
2. You are in charge of the cake for a child's birthday. You have decided the cake will have one candle for each year of their total age. They will only be able to blow out the tallest of the candles. Count how many candles are tallest. Write an algorithm for the above problem and analyse the efficiency of the algorithm.



R.M.K.
GROUP OF
INSTITUTIONS

Content Beyond Syllabus

Unit I Content Beyond Syllabus

Important Data Structures

Graphs:

A graph $G=(V,E)$ is defined by a pair of two sets: A finite set V of items called vertices and a set E of items called edges. If a pair of vertices (u,v) is same as (v,u) in a graph then the graph is called as undirected graph. If pair of vertices (u,v) and (v,u) is not same in a graph then it is called as directed graph or digraph. A graph with every pair of vertices connected by an edge is called complete. A graph with relatively few possible edges missing is called dense; a graph with few edges relative to number of its vertices is called sparse.

Graph representations: Adjacency Matrix and Adjacency List.

Adjacency Matrix: A graph with n vertices is represented by $n \times n$ matrix. The entries of the matrix are 0 and 1. The entry $A[u,v]=0$ indicates there is no edge from u to v and $A[u,v]=1$ if the edge from u to v is present.

Adjacency List: Collection of linked lists, one for each vertex that contains all the vertices adjacent to the particular vertex.

Weighted graph is a graph with numbers assigned its edges. These numbers are called as weights. If this graph is represented by adjacency matrix $A[u,v]=\text{weight of the edge } [u,v]$ and ∞ if there is no edge. Such a matrix is called as weight matrix or cost matrix.

A path from vertex u to vertex v is sequence of adjacent vertices starts with u and ends with v . length of the path is total number of vertices in a vertex sequence defining the path minus 1. A directed path is a sequence of vertices in which every consecutive pair of the vertices is connected by a directed edge. A graph is said to be connected if for every pair of its vertices u and v there is a path from u to v . a graph with no cycles is said to be acyclic.

Trees:

A tree is a connected acyclic graph.

$|E|=|V|-1$ no of edges is no of vertices -1

Rooted trees: for every two vertices in a tree there exists one simple path from one of these vertices to the other. Select an arbitrary vertex in a free tree and it is root.

Ancestors: for any vertex v in a tree, all the vertices on the simple path from the root to that vertex are called ancestors of v .

Parent & child: if (u,v) is the last edge of the simple path from the root to vertex v , u is parent and v is child.

Siblings: vertices that have same parent

Unit I Content Beyond Syllabus

Important Data Structures

Leaf: vertices with no children

Descendants: all the vertices for which a vertex v is an ancestor are said to be descendants of v

Subtree: vertex v with all its descendants is called the subtree.

Depth: length of the simple path from the root to v

Height: length of the longest simple path from root to a leaf.

Ordered trees: rooted tree in which all the children of each vertex are ordered.

Binary tree: ordered tree in which each vertex has no more than two children. Each child is designated as left and right child.

Binary search tree: A number assigned to each parental vertex is larger than the numbers in its left subtree and smaller than its right subtree.

Sets & Dictionaries:

Set: unordered collection of distinct items called elements of the set.

Bit Vector: i^{th} element is 1 iff i^{th} element of U is included in set S .

Dictionary: ordered collection of words. Operations are: searching, adding, and deleting an item.

Assessment Schedule (Proposed Date & Actual Date)

Assessment Schedule

Assessment Tool	Proposed Date	Actual Date	Course Outcome	Program Outcome (Filled Gap)
Assessment I			CO1, CO2	
Assessment II			CO3, CO4	
Model			CO1, CO2, CO3, CO4, CO5	





Prescribed Text Books & Reference

Prescribed Text & Reference Books

Sl. No.	Book Name & Author	Book
1	Anany Levitin, —Introduction to the Design and Analysis of Algorithms , Third Edition, Pearson Education, 2012.	Text Book
2	Ellis Horowitz, Sartaj Sahni and Sanguthevar Rajasekaran, Computer Algorithms/ C++, Second Edition, Universities Press, 2019	Text Book
3	Thomas H.Cormen, Charles E.Leiserson, Ronald L. Rivest and Clifford Stein, —Introduction to Algorithms , Third Edition, PHI Learning Private Limited, 2012.	Reference Book
4	Alfred V. Aho, John E. Hopcroft and Jeffrey D. Ullman, —Data Structures and Algorithms , Pearson Education, Reprint 2006.	Reference Book
5	Harsh Bhasin, —Algorithms Design and Analysis , Oxford university press, 2016	Reference Book
6	S. Sridhar, —Design and Analysis of Algorithms , Oxford university press, 2014.	Reference Book
7	http://nptel.ac.in/	Reference Book
8	https://doc.lagout.org/science/0_Computer%20Science/2_Algorithms/Introduction%20to%20the%20Design%20and%20Analysis%20of%20Algorithms%20%283rd%20ed.%29%20%5BLevitin%202011-10-09%5D.pdf	E-Book

Mini Project Suggestions



R.M.K.
GROUP OF
INSTITUTIONS

Mini Project

1. Climbing stairs

Find the number of different ways to climb an n -stair staircase if each step is either one or two stairs. For example, a 3-stair staircase can be climbed three ways: 1-1-1, 1-2, and 2-1. **(CO1, K3)**

2. Locker doors:

There are n lockers in a hallway, numbered sequentially from 1 to n . Initially, all the locker doors are closed. You make n passes by the lockers, each time starting with locker #1. On the i th pass, $i = 1, 2, \dots, n$, you toggle the door of every i th locker: if the door is closed, you open it; if it is open, you close it. After the last pass, which locker doors are open and which are closed? How many of them are open? **(CO1, K6)**

3. Design an algorithm to find all the common elements in two sorted lists of numbers. For example, for the lists 2, 5, 5, 5 and 2, 2, 3, 5, 5, 7, the output should be 2, 5, 5. What is the maximum number of comparisons your algorithm makes if the lengths of the two given lists are m and n , respectively?

Determine the asymptotic order of growth for the total number of times all the doors are toggled in the locker doors puzzle. **(CO1, K6)**

4. Glove selection:

There are 22 gloves in a drawer: 5 pairs of red gloves, 4 pairs of yellow, and 2 pairs of green. You select the gloves in the dark and can check them only after a selection has been made. What is the smallest number of gloves you need to select to have at least one matching pair in the best case? In the worst case? **(CO1, K4)**

Mini Project

5. . Modern cities rely on real-time traffic management systems to monitor and control traffic flow, detect congestion, and manage traffic signals efficiently. These systems process large volumes of data from sensors, cameras, and GPS devices continuously. Choosing the right algorithms for searching, sorting, and data processing is crucial to ensure minimal response time and smooth traffic flow. **(CO1, K4)**

6. Student-Project Allocation problem (SPA)

In many university departments, students seek to undertake a project in a given field of specialty as part of the upper level of their degree programme. Typically a wide range of available projects is offered, and usually the total number of project places exceeds the number of students, to provide something of a choice. Also, typically each lecturer will offer a variety of projects, but does not necessarily expect that all will be taken up. Each student has preferences over the available projects that he/she finds acceptable, whilst a lecturer will normally have preferences over the students that he/she is willing to supervise. There may also be upper bounds on the number of students that can be assigned to a particular project, and the number of students that a given lecturer is willing to supervise.

Design a Bruteforce algorithm for the problem of allocating students to projects based on these preference lists and capacity constraints and find the efficiency of the same. **(CO1, K6)**

Mini Project

7. Frying hamburgers

There are n hamburgers to be fried on a small grill that can hold only two hamburgers at a time. Each hamburger has to be fried on both sides; frying one side of a hamburger takes 1 minute, regardless of whether one or two hamburgers are fried at the same time.

Consider the following recursive algorithm for executing this task in the minimum amount of time. If $n \leq 2$, fry the hamburger or the two hamburgers together on each side. If $n > 2$, fry any two hamburgers together on each side and then apply the same procedure recursively to the remaining $n - 2$ hamburgers.

- Set up and solve the recurrence for the amount of time this algorithm needs to fry n hamburgers.
- Explain why this algorithm does not fry the hamburgers in the minimum amount of time for all $n > 0$.
- Give a correct recursive algorithm that executes the task in the minimum amount of time.

(CO1, K4)

8. Applications, such as stock market analysis, online gaming leaderboards, or IoT sensor data processing, require fast and efficient computation. Recursive algorithms are often used for tasks like tree traversals, sorting, and dynamic programming problems, but they can be computationally intensive and may affect real-time performance if not optimized. Analyzing their performance mathematically helps in understanding their time and space complexity and predicting scalability. **(CO1, K4)**



Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.